Plug & Pray

Because Hot-Plugging is an Act of Faith

Donde el Scheduler propone y el Segfault dispone

Cátedra de Sistemas Operativos

Trabajo práctico Cuatrimestral

-1C2026 -
Versión 1.0

Índice

Índice 2
Historial de Cambios 4
Objetivos del Trabajo Práctico 5
Características 5
Evaluación del Trabajo Práctico 5
Deployment y Testing del Trabajo Práctico 6
Definición del Trabajo Práctico 7
¿Qué es el trabajo práctico y cómo empezamos? 7
Arquitectura del sistema 8
Aclaración importante 8
Módulo: Kernel Scheduler 9
Lineamiento e Implementación 9
Planificación de Procesos 9
Planificación de Largo Plazo 10
Planificación de Mediano Plazo 10
Planificación de Corto Plazo 10
Desalojo por compactación 10
Mutex 11
Herencia de prioridades 11
IO 11
Sleep 11
STDIN 11
STDOUT 12
Logs mínimos y obligatorios 12
Archivo de Configuración 13
Ejemplo de Archivo de Configuración 13
Módulo: Kernel Memory 14
Lineamiento e Implementación 14
Memoria de Instrucciones y Contextos 14
Contexto de Ejecución 14
Archivos de pseudocódigo 14
Memoria de Usuario o Datos 15
Esquema de memoria y Estructuras 15
Desconexión de un Memory Stick 15
Operaciones 15
Creación de Proceso 15
Creación de Segmento 15
Algoritmos de selección de huecos 15
Compactación 16
Eliminación de Segmento 16
Suspensión de Proceso 16
Des-suspensión de Proceso 16
Finalización de Proceso 16
Lectura de datos 16
Escritura de datos 17
Logs mínimos y obligatorios 17
Archivo de Configuración 17

Ejemplo de Archivo de Configuración 18
Módulo: CPU **19**
Lineamiento e Implementación 19
Registros de la CPU 19
Ciclo de Instrucción 20
Fetch 20
Decode 20
Ejemplos de instrucciones a interpretar 20
Execute 21
Check Interrupt 22
MMU 22
Logs mínimos y obligatorios 22
Archivo de Configuración 23
Ejemplo de Archivo de Configuración 23
Módulo: Memory Stick **24**
Lineamiento e Implementación 24
Operaciones 24
Lectura de datos 24
Logs mínimos y obligatorios 24
Archivo de Configuración 25
Ejemplo de Archivo de Configuración 25
Módulo: IO **26**
Lineamiento e Implementación 26
STDIN 26
STDOUT 26
SLEEP 26
Logs mínimos y obligatorios 26
Archivo de Configuración 27
Ejemplo de Archivo de Configuración 27
Módulo: SWAP **28**
Lineamiento e Implementación 28
Operaciones 28
Lectura de bloque 28
Logs mínimos y obligatorios 28
Archivo de Configuración 29
Ejemplo de Archivo de Configuración 29
Descripción de las entregas **30**
Check de Control Obligatorio 1: Conexión inicial 30
Check de Control Obligatorio 2: Planificación 30
Check de Control Obligatorio 3: CPU Completa y Memoria 31
Entregas Finales 31

Historial de Cambios

v1.0 (07/04/2026) Publicación del enunciado

Objetivos del Trabajo Práctico

Mediante la realización de este trabajo se espera que el alumno:

- Adquiera conceptos prácticos del uso de las distintas herramientas de programación e interfaces (APIs) que brindan los sistemas operativos.
- Entienda aspectos del diseño de un sistema operativo.
- Afirme diversos conceptos teóricos de la materia mediante la implementación práctica de algunos de ellos.
- Se familiarice con técnicas de programación de sistemas, como el empleo de makefiles, archivos de configuración y archivos de log.

Debido al fin académico del trabajo práctico, los conceptos que se verán reflejados son, en general, versiones simplificadas o alteradas de los componentes reales de hardware y de sistemas operativos vistos en las clases, a fin de resaltar aspectos de diseño o simplificar su implementación.

Invitamos a los alumnos a leer las notas y comentarios al respecto que haya en el enunciado, reflexionar y discutir con sus compañeros, ayudantes y docentes al respecto.

Características

- Modalidad: grupal (5 integrantes ± 0) y obligatorio
- Fecha de comienzo: 07/04/2026
- Fecha de primera entrega: 11/07/2026
- Fecha de segunda entrega: 18/07/2026
- Fecha de tercera entrega: 01/08/2026
- Lugar de corrección: Laboratorio de Sistemas - Medrano.

Evaluación del Trabajo Práctico

El trabajo práctico consta de una evaluación en 2 etapas.

La primera etapa consistirá en las pruebas de los programas desarrollados en el laboratorio. Previo a la evaluación final se subirán una serie de pruebas para que los alumnos puedan validar el correcto funcionamiento de su trabajo práctico. Queda aclarado que para que un trabajo práctico sea considerado evaluable, el mismo debe proporcionar logs de su funcionamiento de la forma más clara posible, para ello se les proveerá en cada módulo un listado de logs mínimos y obligatorios.

La segunda etapa se dará en caso de aprobada la primera y constará de un coloquio individual, con el objetivo de validar y afianzar la aplicación de los conocimientos adquiridos durante el desarrollo del trabajo práctico y definir la nota de cada uno de los integrantes del grupo, por lo que se recomienda que la carga de trabajo se distribuya de la manera más equitativa posible.

Cabe aclarar que el trabajo equitativo no asegura la aprobación de la totalidad de los integrantes, sino que cada uno tendrá que defender y explicar tanto teórica como prácticamente lo desarrollado y aprendido a lo largo de la cursada.

La defensa del trabajo práctico (o coloquio) consta de la relación de lo visto durante la teoría con lo implementado. De esta manera, una implementación que contradiga lo visto en clase o lo escrito en el documento *es motivo de desaprobación del trabajo práctico*. Esta etapa al ser la conclusión del todo el trabajo realizado durante el cuatrimestre *no es recuperable*.

Deployment y Testing del Trabajo Práctico

Al tratarse de una plataforma distribuida, los procesos involucrados deberán ser ejecutados en diferentes computadoras. La cantidad de computadoras involucradas y la distribución de los diversos procesos en estas será definida en cada uno de los tests de la evaluación y es posible cambiar la misma en el momento de la evaluación. Es responsabilidad del grupo automatizar el despliegue de los diversos procesos con sus correspondientes archivos de configuración para cada uno de los diversos tests a evaluar.

Todo esto estará detallado en el documento de pruebas que se publicará cercano a la fecha de Entrega Final. Archivos y programas de ejemplo se pueden encontrar en el repositorio de la cátedra.

Finalmente, es mandatoria la lectura y entendimiento de las Normas del Trabajo Práctico donde se especifican todos los lineamientos de cómo se desarrollará la materia durante el cuatrimestre.

Definición del Trabajo Práctico

Esta sección se compone de una introducción y definición de carácter global sobre el trabajo práctico. Posteriormente se explicarán por separado cada uno de los distintos módulos que lo componen, pudiéndose encontrar los siguientes títulos:

- **Lineamiento e Implementación**: Contendrá la definición funcional y aspectos de implementación técnica obligatorios del módulo en cuestión. La no inclusión de alguno de los puntos especificados en este título puede conllevar a la desaprobación del trabajo práctico.

- **Archivos de Configuración**: Se describirán los parámetros mínimos requeridos para ajustar el comportamiento de cada módulo ante cada prueba, sin recompilar. Durante la evaluación, deberá alcanzar con detener la ejecución, modificar el archivo de configuración y volver a ejecutar el módulo. En caso de que el grupo requiera de algún parámetro extra, podrá agregarlo.

Cada módulo contará con un listado de **logs mínimos y obligatorios** los cuales deberán realizarse utilizando la biblioteca de so-commons-library provista por la cátedra y los mismos deberán estar como LOG_LEVEL_INFO, pudiendo ser extendidos por necesidad del grupo utilizando LOG_LEVEL_DEBUG. En la descripción de los **logs mínimos y obligatorios**, se utiliza la notación de <> para encerrar los valores que irán variando de acuerdo a la ejecución, pero los caracteres menor y mayor (< >) no deberán estar presentes en el log final.

En caso de no cumplir con los logs mínimos y/o no guardarlos en archivo, *se considerará que el TP no es apto para ser evaluado* y por consecuencia el mismo estará *desaprobado*.

Cabe destacar que en ciertos puntos de este enunciado se explicarán exactamente cómo deben ser las funcionalidades a desarrollar, mientras que en otros no se definirá específicamente, quedando su implementación a decisión y definición del equipo. Se recomienda en estos casos siempre consultar en el foro de github, dado que las justificaciones deberán ser expuestas en el coloquio.

¿Qué es el trabajo práctico y cómo empezamos?

El objetivo del trabajo práctico consiste en desarrollar una solución que permita la simulación de un sistema distribuido, donde los grupos tendrán que planificar Procesos, resolver peticiones al sistema, y administrar de manera adecuada una memoria bajo los esquemas explicados en sus correspondientes módulos.

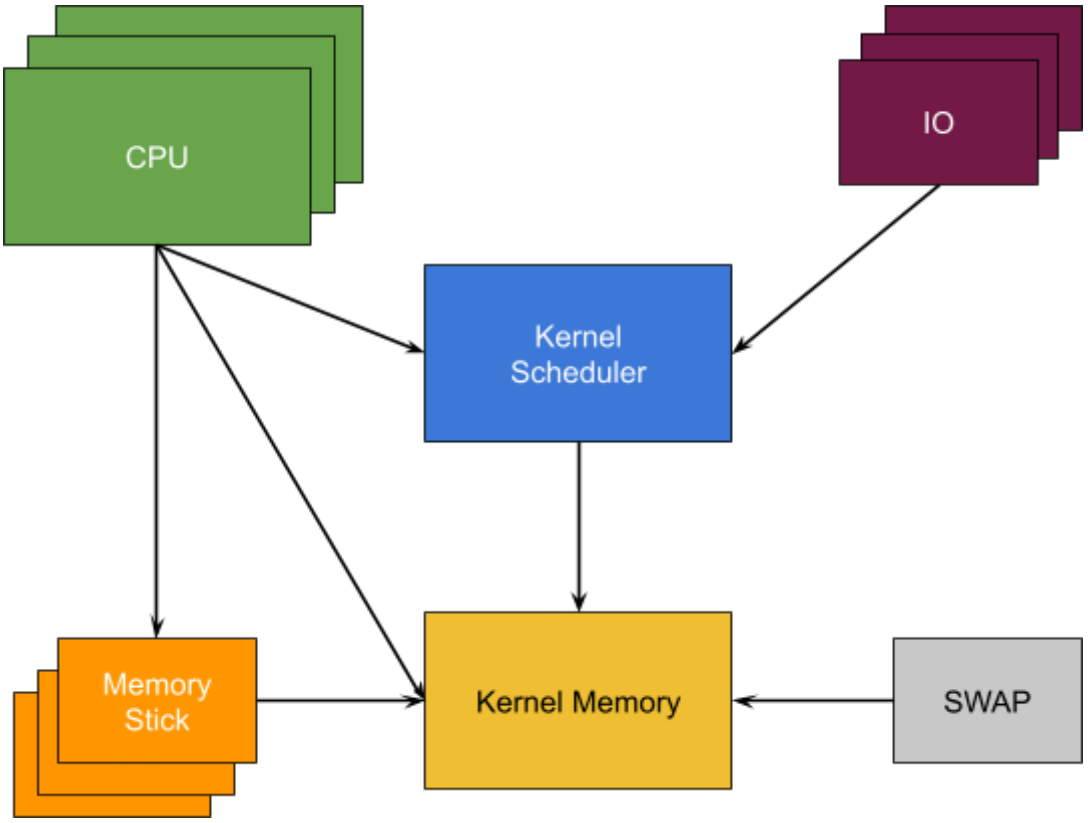
Para el desarrollo del mismo se decidió la creación de un sistema bajo la metodología Iterativa Incremental donde se solicitarán en una primera instancia la implementación de ciertos módulos para luego poder realizar una integración total con los restantes.

Recomendamos seguir el lineamiento de los distintos puntos de control que se detallan al final de este documento para su desarrollo. Estos puntos están planificados y estructurados para que sean desarrollados a medida y en paralelo a los contenidos que se ven en la parte teórica de la materia.

Cabe aclarar que esto es un lineamiento propuesto por la cátedra y no implica impedimento alguno para el alumno de realizar el desarrollo en otro orden diferente al especificado.

Arquitectura del sistema

Este trabajo práctico correrá una serie de módulos los cuales deberán ser capaces de correr en diferentes computadoras y/o máquinas virtuales.



NOTAS:

- El sentido de las flechas indica dependencias entre módulos. Ejemplo: al momento de iniciar la ejecución del Kernel Scheduler es necesario contar con el Módulo Kernel Memory ya iniciado.
- Cada uno de estos Módulos será un programa realizado en lenguaje C, compilado y ejecutado en la Máquina Virtual, por lo que será un proceso real del sistema operativo.
- A lo largo de este enunciado llamaremos Proceso (con P mayúscula) a los *procesos simulados* que se planificarán y ejecutarán dentro de la implementación de este trabajo práctico.

Aclaración importante

Desarrollar únicamente temas de conectividad, serialización, sincronización o los módulos IO, SWAP o Memory Stick son insuficientes para poder entender y aplicar los distintos conceptos de la materia. Dicho caso será un motivo de desaprobación directa.

Módulo: Kernel Scheduler

Dentro de este TP, el Kernel Scheduler será el módulo encargado de gestionar la planificación de los Procesos a lo largo de las diferentes pruebas que se tengan que realizar.

Lineamiento e Implementación

El Kernel Scheduler contará con un primer parámetro que será el Path al Proceso Inicial, el cual será el PID 0 del sistema, que siempre tendrá prioridad máxima, y a partir del cual se crearán los demás Procesos.

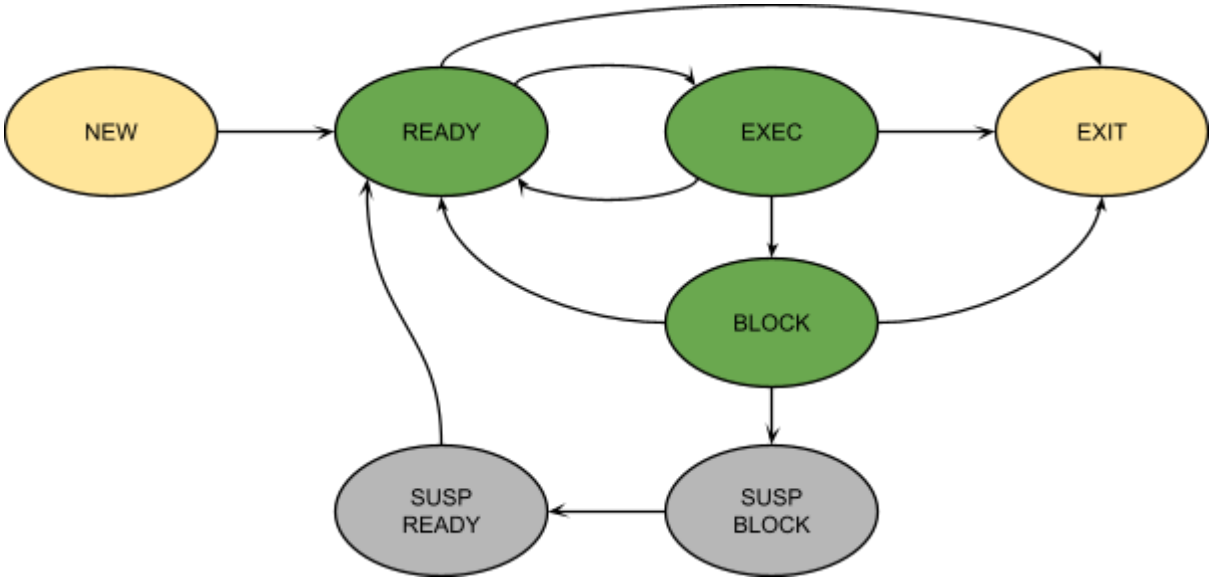
```
➜ ~ ./bin/kernel_scheduler [Archivo Config] [Path Proceso Inicial]
```

Al iniciar el módulo, el mismo se deberá conectar con el módulo Kernel Memory y, una vez que establezca dicha conexión, deberá crear un servidor *multihilo* para atender de forma concurrente las peticiones de las CPUs y de los módulos de IO.

A lo largo de la ejecución se podrán desconectar y conectar nuevas CPUs, por lo que se deberá mantener la escucha a nuevas conexiones siempre activa.

Planificación de Procesos

Una vez creado el servidor para escuchar las conexiones, el Kernel Scheduler deberá cumplir con las tareas de planificación de largo, mediano y corto plazo detalladas más adelante. La implementación de los mismos es una decisión de diseño que deberá tomar el grupo, pero deberá ser capaz de gestionar un modelo de 7 estados:



Planificación de Largo Plazo

Se encargará de ingresar Procesos de NEW a READY. Dado que los Procesos inician sin memoria asignada, este pasaje se podrá realizar sin ninguna limitación.

En algún momento de la ejecución es posible que se reciba una notificación del Kernel Memory de que se detectó corrupción en una parte de la memoria. Ante este evento, se deberán finalizar todos los Procesos y finalizar el Kernel Scheduler con motivo de Blue Screen of Death (BSOD).

Planificación de Mediano Plazo

Se encargará de las transiciones entre los estados BLOCK y SUSP. BLOCK o entre los estados SUSP. READY y READY.

Para realizar la transición del estado BLOCK a SUSP. BLOCK, por cada Proceso que entre al estado BLOCK, se deberá esperar un tiempo determinado por archivo de configuración. Si al transcurrir dicho tiempo el Proceso aún continúa en estado BLOCK, se lo deberá mover al estado SUSP. BLOCK.

En caso de que se libere memoria, se agregue un memory stick nuevo, o se compacte la misma, se recorrerán todos los Procesos suspendidos por orden de prioridad y se irán des-suspendiendo si cuentan con espacio disponible para re-crear todos sus segmentos *sin disparar una compactación*. En caso de contar con más de un Proceso suspendido con la misma prioridad, se evaluará primero el que lleve mayor tiempo suspendido.

Planificación de Corto Plazo

Se encargará de gestionar las transiciones entre los estados READY y EXEC.

Vamos a estar utilizando uno de los tres posibles algoritmos de planificación definidos, los cuales van a ser **FIFO**, **RR** y **Colas Multinivel** (no retroalimentadas). El algoritmo a utilizar será elegible por archivo de configuración y no cambiará a lo largo de una prueba.

La planificación basada en Colas Multinivel definirá una cola por cada nivel de prioridad dentro del sistema. El algoritmo (FIFO o RR) a utilizar dentro de cada cola estará especificado dentro del archivo de configuración. Las prioridades comenzarán en 0, y no se deberán planificar Procesos que tengan un nivel de prioridad fuera del rango definido dentro del archivo de configuración. Por ejemplo, si se definen 4 colas en la configuración, solamente se planificarán prioridades de la 0 a la 3.

En los algoritmos de FIFO y RR la prioridad no tendrá injerencia alguna, mientras que en el algoritmo de Colas Multinivel, al momento de llegar un Proceso, y si el desalojo entre colas está habilitado, se deberá validar si se encuentra ejecutando un Proceso con menor prioridad. En caso de que así sea, se deberá desalojar al Proceso menos prioritario para darle lugar al más prioritario que acaba de llegar.

Desalojo por compactación

Dentro de la ejecución normal de los diferentes Procesos puede existir la situación en la cual un Proceso, al momento de intentar crear un segmento, dispare la compactación de la memoria. Cuando esto suceda, se recibirá el pedido desde el módulo Kernel Memory para que se desalojen todos los Procesos antes de iniciar la compactación.

Al recibir este pedido, el Kernel Scheduler enviará la solicitud de desalojo a todas las CPU y no enviará ningún otro Proceso a ejecutar hasta que reciba la confirmación por parte del Kernel Memory que finalizó la compactación. Los Procesos desalojados, excepcionalmente, serán colocados al principio de la cola de READY.

Una vez finalizada la misma se procederá a replanificar todos los Procesos de acuerdo al algoritmo definido.

Mutex

Dentro del módulo también se gestionarán los semáforos de tipo Mutex que tendrá nuestro lenguaje de scripting creado con fines didácticos.

Estos semáforos tendrán la particularidad de que, al ser de tipo Mutex, solo un Proceso los podrá tener tomado al mismo tiempo y el resto de Procesos deberán aguardar a ser atendidos.

Herencia de prioridades

A lo largo de la ejecución puede darse la situación en la que un Proceso de prioridad baja tome un Mutex libre, y que luego de esto, un Proceso con mayor prioridad intente tomar el mismo mutex. Al estar previamente tomado por el Proceso de menor prioridad, este segundo queda bloqueado.

Ante este evento, donde un Proceso de una prioridad mayor queda bloqueado a la espera de un Proceso con prioridad menor, este último heredará temporalmente la prioridad del Proceso con mayor prioridad que se encuentre bloqueado esperando el Mutex. Una vez que el Proceso de baja prioridad libere el Mutex, volverá a tener su prioridad original.

IO

Las IO dentro del entorno de nuestro trabajo práctico pueden ser 3: SLEEP, STDIN y STDOUT. En todos los casos, al momento de que llegue la petición de la CPU, el proceso pasará del estado EXEC al estado BLOCK y el Kernel Scheduler deberá enviar un nuevo Proceso a ejecutar si lo hubiera.

Sleep

Para las IO de tipo Sleep se recibirá de la CPU un tiempo en milisegundos que deberá estar el Proceso en la IO de tipo Sleep.

STDIN

Para las IO de tipo STDIN, se recibirán por parte de la CPU un tamaño total a leer y la dirección lógica donde se deberá guardar el contenido leído.

Se deberá solicitar a la IO de tipo STDIN que le solicite al usuario ingresar el total a leer y luego, cuando este retorne los caracteres leídos, se deberán enviar al Módulo Kernel Memory para que este las escriba en donde corresponda.

STDOUT

En el caso de las IO de tipo STDOUT, el Kernel Scheduler recibirá de parte de la CPU la dirección lógica y el tamaño a leer. Una vez que este le retorne todos los bytes, deberá pedirle esta información al Kernel Memory para luego, enviársela a la IO de tipo STDOUT.

Logs mínimos y obligatorios

Conexión a Kernel Memory: "`## Conectado a Kernel Memory`"

Conexión de CPU: "`## CPU <ID CPU> Conectada`"

Creación de Proceso: "`## (<PID>) Se crea el proceso - Estado: NEW`"

Syscall recibida: "`## (<PID>) - Solicitó syscall: <NOMBRE_SYSCALL>`"

Cambio de Estado: "`## (<PID>) Pasa del estado <ESTADO_ANTERIOR> al estado <ESTADO_ACTUAL>`"

Fin de IO: "`## (<PID>) finalizó IO y pasa a READY / SUSP. READY`"

Mutex Tomado: "`## (<PID>) Toma el Mutex <NOMBRE_MUTEX>`"

Mutex Liberado: "`## (<PID>) Libera el Mutex <NOMBRE_MUTEX>`"

Cambio de prioridad de Proceso: "`## <PID> Cambio de prioridad: <PRIORIDAD_ANTERIOR> - <PRIORIDAD_NUEVA>`"

Desalojo por fin de Quantum: "`## (<PID>) - Desalojado por fin de quantum`"

Desalojo por Cola más prioritaria: "`## (<PID>) Prioridad: <PRIORIDAD_DESALOJADO> - Desalojado por cola más prioritaria por el proceso <PID> con prioridad <PRIORIDAD_NUEVA>`"

Inicio compactación: "`## Inicio de compactación`"

Fin compactación: "`## Fin de compactación`"

Fin de Proceso: "`## (<PID>) finalizó su ejecución con motivo de <MOTIVO>`"

Archivo de Configuración

Campo	Tipo	Descripción
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level_from_string()
PLANIFICATION_ALGORITHM	String	Indica el algoritmo de planificación a utilizar, los valores posibles son FIFO, RR y CMN.

Campo	Tipo	Descripción
QUEUES_ALGORITHMS	Lista	Lista de algoritmos a utilizar cuando el algoritmo sea CMN.
RR_QUANTUM	Número	Tiempo en milisegundos que se debe esperar antes de finalizar el Quantum de una CPU.
QUEUE_PREEMPTION	String	Desalojo entre colas habilitado, valores posibles TRUE / FALSE
SUSPENSION_TIMEOUT	Número	Tiempo en milisegundos de espera antes de suspender un proceso que está en IO.

Ejemplo de Archivo de Configuración

```
LOG_LEVEL=INFO
PLANIFICATION_ALGORITHM=CMN
QUEUES_ALGORITHMS=[FIFO,RR,RR,FIFO,RR,FIFO]
RR_QUANTUM=1500
QUEUE_PREEMPTION=TRUE
SUSPENSION_TIMEOUT=35000
```

Módulo: Kernel Memory

Este módulo será el encargado de gestionar la asignación de memoria a lo largo de los diferentes Memory Sticks y del SWAP.

Lineamiento e Implementación

Al iniciar creará un servidor *multihilo* que se encargará de gestionar de manera concurrente las peticiones del Kernel Scheduler y las CPUs, así como también esperará las conexiones de los diferentes Memory Sticks y del SWAP.

A lo largo de la ejecución se podrán conectar nuevas CPUs y Memory Sticks, por lo que se deberá mantener la escucha a nuevas conexiones siempre activa.

```
➜ ~ ./bin/kernel_memory [Archivo Config]
```

Memoria de Instrucciones y Contextos

En esta sección el Módulo Kernel Memory contendrá las instrucciones y los contextos de ejecución que le serán solicitadas por las diferentes CPUs.

Contexto de Ejecución

Por cada PID del sistema se deberá almacenar un *contexto de ejecución*, el cual consiste en una copia de todos los registros de la CPU y la tabla de segmentos asociada a dicho Proceso.

Este módulo deberá mantener el contexto de ejecución de cada Proceso del sistema. Deberá ser capaz de enviar y/o recibir el contexto de ejecución hacia/desde el módulo CPU cuando este se lo solicite.

Archivos de pseudocódigo

Los archivos de pseudocódigo serán archivos de texto, los cuales contendrán las instrucciones que ejecutan las CPUs, estando estas separadas por el caracter '\n'.

Por cada PID del sistema se tendrá un archivo de pseudocódigo, para lo que cada grupo deberá implementar una estructura que le permita asociar qué PID tiene asociado qué archivo de pseudocódigo.

El Kernel Memory deberá ser capaz de enviar la instrucción correspondiente a cada pedido de la CPU sin haber una restricción específica de cómo se deben obtener las mismas (queda a criterio de cada grupo la implementación de esta sección).

Memoria de Usuario o Datos

Esquema de memoria y Estructuras

La memoria de usuario de nuestro Kernel Memory utilizará el esquema de segmentación pura, por lo que se deberá almacenar para cada PID del sistema una estructura que permita conocer todos los segmentos correspondientes a dicho PID con su base y su límite.

Para el almacenamiento de los datos se utilizarán los módulos Memory Stick, los cuales al momento de conectarse deberán informarle su tamaño al módulo Kernel Memory. Una vez recibido el tamaño, se deberá añadir al final de la lista de Memory Sticks conectados, ampliando el tamaño máximo de la memoria total.

Al momento de que se conecte un nuevo Memory Stick y se amplíe el tamaño total de la memoria, se deberá notificar al Kernel Scheduler que se dispone de más memoria.

Desconexión de un Memory Stick

Existe la posibilidad de que en medio de una ejecución un Memory Stick se desconecte. Dado el caso, se deberá notificar al Kernel Scheduler que la memoria se encuentra corrupta.

Operaciones

El Kernel Memory, al ser el módulo que gestiona la memoria del sistema y de los Procesos, puede aceptar una serie de operaciones que describiremos a continuación.

Creación de Proceso

Para esta operación se va a recibir un PID y el path de un archivo de instrucciones, relativo[1] al path base configurado por archivo de configuración. Con los datos recibidos, el Módulo Kernel Memory deberá crear el contexto de ejecución del Proceso, inicializando todos los registros asociados con el valor 0.

Creación de Segmento

Al momento de crear un segmento el Kernel Memory recibirá como parámetro el PID del Proceso, un ID de segmento y el tamaño que debe tener el nuevo segmento.

Algoritmos de selección de huecos

Al momento de crear un nuevo segmento es posible que se tenga más de 1 hueco disponible, por lo que se debe poder variar el algoritmo con el cual se va a estar seleccionando qué hueco va a ser el candidato para recibir el nuevo segmento creado.

Los Algoritmos a implementar son **Best Fit** y **Worst Fit**, y la elección del mismo se hará por medio del archivo de configuración del Kernel Memory.

---

[1] Rutas Relativas y Absolutas - Guía

Compactación

Es posible que al momento de intentar crear un Segmento se cuente con el espacio disponible en la memoria pero la misma no se encuentre toda contigua, por lo que se le deberá notificar al Kernel Scheduler que es necesario realizar una compactación y que, para ello, debe desalojar todas las CPUs.

Una vez recibida la confirmación por parte del Kernel de que se desalojaron las CPUs comenzará el proceso de compactación, en el cual se tendrán que reordenar todos los segmentos de memoria al principio de la misma. Esto puede resultar en que algunos segmentos se muevan parcial o totalmente de Memory Stick, debiendo actualizar las tablas de segmentos de todos los Procesos.

Eliminación de Segmento

Al momento de eliminar un segmento se recibirá como parámetro el PID y el ID del segmento a eliminar y se procederá simplemente a eliminar dicha entrada en la tabla de segmentos del proceso. Por último, se marcará ese espacio como libre en la memoria.

Suspensión de Proceso

Al momento de suspender un Proceso se deberán mover todos sus segmentos de los Memory Sticks a bloques dentro del Módulo SWAP. Para ello se irán moviendo de a 1 segmento y se liberará la memoria una vez que el mismo se encuentra copiado.

Es responsabilidad de cada grupo definir las estructuras necesarias para poder saber en qué bloques se encuentra cada segmento. Los bloques de SWAP se asignan por segmento, por lo que *no puede existir un bloque que contenga información de 2 segmentos distintos*.

Des-suspensión de Proceso

Al momento de des-suspender un Proceso se deberán restaurar todos los segmentos que se encuentran almacenados en SWAP a la memoria principal. Para ello se deberá regenerar nuevamente la tabla de segmentos que se encuentra en el contexto de ejecución del Proceso.

Finalización de Proceso

Esta operación se utiliza para finalizar un Proceso, por lo que solamente va a recibir un PID y a partir de este PID recibido se deberán liberar todos los segmentos asociados al Proceso y todas las estructuras asociadas al mismo.

Lectura de datos

Esta operación va estar asociada a un pedido directo del Kernel Scheduler, que va a oficiar de intermediario entre la IO de tipo STDOUT y el Kernel Memory, pidiéndole a este último una cantidad de bytes determinada, que se encuentra almacenada en uno o varios Memory Sticks.

Va a recibir como parámetros una dirección lógica y el tamaño a leer, por lo que deberá realizar la traducción de la dirección lógica a física para llevar a cabo la operación.

Escritura de datos

Esta operación va a estar asociada a un pedido directo del Kernel Scheduler que va a oficiar de intermediario entre la IO de tipo STDIN y el Kernel Memory, pidiéndole a este último que escriba una serie de bytes en uno o varios Memory Sticks.

Va a recibir como parámetros una dirección lógica y una cantidad de bytes a escribir, por lo que deberá realizar la traducción de la dirección lógica a física para llevar a cabo la operación.

Logs mínimos y obligatorios

Conexión de CPU: "`## CPU <ID CPU> Conectada`"

Conexión de Memory Stick: "`## Memory Stick de <TAMAÑO> bytes Conectada`"

Conexión de Kernel Scheduler: "`## Kernel Scheduler Conectado - FD del socket: <FD_DEL_SOCKET>`"

Creación de Proceso: "`## PID: <PID> - Proceso Creado`"

Obtener instrucción: "`## PID: <PID> - Obtener instrucción: <PC> - Instrucción: <INSTRUCCIÓN> <...ARGS>`"

Creación de Segmento: "`## PID: <PID> - Segmento Creado <ID_SEGMENTO> - Tamaño: <TAMAÑO_SEGMENTO>`"

Escritura / lectura en espacio de usuario: "`## PID: <PID> - <Escritura/Lectura> - Dir. Física: <DIRECCIÓN_FÍSICA> - Tamaño: <TAMAÑO>`"

Archivo de Configuración

Campo	Tipo	Descripción
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level_from_string()
SEGMENT_MAX_SIZE	Numérico	Tamaño máximo que puede tener cualquier segmento
ALLOCATION_STRATEGY	String	Indica el algoritmo a utilizar para seleccionar un hueco libre, los valores pueden ser: BEST o WORST
INSTRUCTION_DELAY	Número	Tiempo en milisegundos que se debe esperar antes de contestar una petición de instrucción
COMPACTION_DELAY	Número	Tiempo en milisegundos que se debe esperar

Campo	Tipo	Descripción
		antes de dar por finalizada la compactación
SCRIPTS_BASEPATH	String	Path donde se encuentran los archivos de pseudocódigo

Ejemplo de Archivo de Configuración

```
LOG_LEVEL=INFO
SEGMENT_MAX_SIZE=256
ALLOCATION_STRATEGY=BEST
INSTRUCTION_DELAY=500
COMPACTION_DELAY=30000
SCRIPTS_BASEPATH=/home/utnso/plug-n-pray-pruebas
```

Módulo: CPU

El módulo CPU, en el contexto de nuestro Trabajo Práctico, simulará los pasos del ciclo de instrucción de una CPU real simplificada.

Lineamiento e Implementación

Al momento de ejecutar un CPU se debe indicar por argumento cuál es su "identificador" para poder identificarla de los demás.

```
➜ ~ ./bin/cpu [Archivo Config] [Identificador]
```

El módulo **CPU** es el encargado de interpretar y ejecutar las instrucciones recibidas por parte del **Kernel Memory**. Para ello, ejecutará un ciclo de instrucción simplificado que cuenta con los pasos: Fetch, Decode, Execute y Check Interrupt.

Para cada CPU se deberá tener un archivo de log y archivo de configuración independientes, que deberán contar con el identificador pasado por parámetro para poder identificar a qué CPU corresponde el archivo de log que se está leyendo. Las CPUs deberán conectarse al **Kernel Scheduler**, al **Kernel Memory** y a los diferentes **Memory Sticks**. Estos últimos se podrán ir conectando a lo largo de la ejecución, por lo que es importante que el grupo defina una estrategia entre el Kernel Memory y las CPUs para que éstas conozcan todos los Memory Sticks que existen cuando sea necesario.

Una vez establecidas todas las conexiones, la CPU se quedará a la espera de recibir un **PID** de parte del Kernel Scheduler. Una vez recibido, la CPU deberá solicitarle al Kernel Memory el contexto de ejecución y comenzar el primer ciclo de instrucción.

A la hora de ejecutar instrucciones que requieran interactuar directamente con la Memoria, tendrá que traducir las *direcciones lógicas* (propias del proceso) a *direcciones físicas* (propias de la memoria). Para ello simulará la existencia de una MMU.

Registros de la CPU

En la implementación de nuestra CPU, se utilizará una serie de registros para poder modelar la operatoria de una CPU real simplificada; es decir, vamos a contar con registros similares a los vistos en Arquitectura de Computadores y algunos registros creados por nosotros mismos a fin de poder facilitar las pruebas.

En la siguiente tabla está el detalle de los registros que deberá tener nuestra CPU, en la cual estará detallado el tamaño del mismo y qué tipo de dato se recomienda para su implementación:

Registro	Tamaño	Tipo de Dato	Descripción
PC[2]	4 bytes	uint32_t	Program Counter, indica la próxima instrucción a ejecutar

[2] En caso de que se realice una modificación de este registro en una operación, se omitirá el paso de sumar 1 al final del ciclo de instrucción

Registro	Tamaño	Tipo de Dato	Descripción
AX	1 byte	uint8_t	Registro Numérico de propósito general
BX	1 byte	uint8_t	Registro Numérico de propósito general
CX	1 byte	uint8_t	Registro Numérico de propósito general
DX	1 byte	uint8_t	Registro Numérico de propósito general
EAX	4 bytes	uint32_t	Registro Numérico de propósito general
EBX	4 bytes	uint32_t	Registro Numérico de propósito general
ECX	4 bytes	uint32_t	Registro Numérico de propósito general
EDX	4 bytes	uint32_t	Registro Numérico de propósito general
SI	4 bytes	uint32_t	Contiene la dirección lógica de memoria de origen
DI	4 bytes	uint32_t	Contiene la dirección lógica de memoria de destino

Ciclo de Instrucción

Fetch

La primera etapa del ciclo consiste en buscar la próxima instrucción a ejecutar. En este trabajo práctico cada instrucción deberá ser pedida al módulo Kernel Memory utilizando el *Program Counter* (también llamado *Instruction Pointer*) que representa el número de instrucción a buscar relativo al Proceso en ejecución.

Decode

Esta etapa consiste en interpretar qué instrucción es la que se va a ejecutar y si la misma requiere de una traducción de dirección lógica a dirección física.

Ejemplos de instrucciones a interpretar

```
NOOP
SET AX 6
SET BX 2
SET PC 5
SUM AX BX
SUB AX BX
JNZ AX 4
COPY_MEM AX
MOV_IN EDX
MOV_OUT EDX
```

```
MUTEX_CREATE MUTEX_1
MUTEX_LOCK MUTEX_1
MUTEX_UNLOCK MUTEX_1
MEM_ALLOC 0 64
MEM_FREE 0
SLEEP 25000
STDOUT AX BX
STDIN CX DX
INIT_PROC proceso1 3
EXIT
```

Las instrucciones detalladas previamente son a modo de ejemplo, su ejecución no necesariamente sigue lógica alguna ni funcionamiento correcto. Al momento de realizar las pruebas, ninguna instrucción contendrá errores sintácticos ni semánticos.

Execute

En este paso se deberá ejecutar lo correspondiente a cada instrucción:

- **NOOP**: Representa la instrucción No Operation, es decir, va a consumir solamente el tiempo del ciclo de instrucción.
- **SET** (Registro, Valor)**:** Asigna al registro el valor pasado como parámetro.
- **MOV_IN** (Registro Datos): Lee el valor de memoria correspondiente a la Dirección Lógica que se encuentra en el Registro **SI** y lo almacena en el Registro Datos.
- **MOV_OUT** (Registro Datos)**:** Lee el valor del Registro Datos y lo escribe en la dirección física de memoria obtenida a partir de la Dirección Lógica almacenada en el Registro **DI**.
- **SUM** (Registro Destino, Registro Origen): Suma al Registro Destino el Registro Origen y deja el resultado en el Registro Destino.
- **SUB** (Registro Destino, Registro Origen): Resta al Registro Destino el Registro Origen y deja el resultado en el Registro Destino.
- **JNZ** (Registro, Instrucción): Si el valor del registro es distinto de cero, actualiza el *program counter* al número de instrucción pasada por parámetro.
- **COPY_MEM** (Registro Tamaño): Toma la dirección apuntada por el registro SI y copia la cantidad de bytes indicadas en el *registro tamaño* a la posición de memoria apuntada por el registro DI.

Las siguientes instrucciones se considerarán *Syscalls*, ya que las mismas no pueden ser resueltas por la CPU y depende de la acción del Kernel Scheduler para su realización. A diferencia de la vida real donde la llamada es a una única instrucción, para simplificar la comprensión de los scripts, vamos a utilizar un nombre diferente para cada *Syscall*. La descripción de lo que hace cada una va a estar detallada en el Kernel Scheduler.

- **MUTEX_CREATE** (Nombre Mutex)
- **MUTEX_LOCK** (Nombre Mutex)
- **MUTEX_UNLOCK** (Nombre Mutex)
- **MEM_ALLOC** (Id del Segmento, Tamaño)

- **MEM_FREE** (Id del Segmento)
- **SLEEP** (Tiempo)
- **STDOUT** (Registro Dirección Lógica, Registro Tamaño)
- **STDIN** (Registro Dirección Lógica, Registro Tamaño)
- **INIT_PROC** (Archivo de instrucciones, Prioridad)
- **EXIT**

Es importante tener en cuenta que, al finalizar el ciclo de instrucción, el Program Counter (PC) deberá ser actualizado sumándole 1, siempre y cuando este no haya sido modificado por la instrucción ejecutada anteriormente.

Check Interrupt

En este momento, se deberá chequear si el **Kernel Scheduler** nos envió una *interrupción* al PID que se está ejecutando. En caso afirmativo, se actualiza el contexto de ejecución en el Kernel Memory y se le devuelve el PID al Kernel Scheduler con el *motivo* de la interrupción. Caso contrario, se descarta la interrupción.

MMU

A la hora de traducir **direcciones lógicas a físicas**, la CPU debe tomar en cuenta que el esquema de la memoria de este sistema es de **Segmentación**, por lo tanto, las direcciones lógicas se interpretarán de la siguiente manera:

[N° Segmento | Desplazamiento]

Estas traducciones, a diferencia de lo que se hace en los ejercicios prácticos que se ven en clases y se toman en los parciales, no se realizarán en binario, ya que es más cómodo utilizar números enteros en sistema decimal para ver los valores. Por lo tanto, la operatoria sería más parecida a la siguiente:

num_segmento = *floor*(dir_logica / **tam_max_segmento**)

desplazamiento_segmento = dir_logica *%* **tam_max_segmento**

En algunas traducciones es posible que una petición de lectura o escritura abarque más de 1 Memory Stick. Será responsabilidad del grupo dividir estas peticiones de acuerdo a los diferentes Memory Sticks involucrados, teniendo en cuenta que las direcciones físicas de los Memory Stick arrancan a partir del 0. Luego se deberá consolidar el resultado de la operación para que la misma se considere como una única operación.

En caso de que el desplazamiento dentro del segmento (**desplazamiento_segmento**) sumado al tamaño a leer / escribir sea mayor al tamaño del segmento, deberá devolverse el Proceso al **Kernel Scheduler** para que este lo finalice con motivo de **Error: Segmentation Fault (SEG_FAULT)**.

Logs mínimos y obligatorios

Fetch Instrucción: “`## PID: <PID> - FETCH - Program Counter: <PROGRAM_COUNTER>`”.

Interrupción Recibida: “## Interrupción recibida”.

Instrucción Ejecutada: “## PID: <PID> - Ejecutando: <INSTRUCCION> - <PARAMETROS>”.

Lectura/Escritura Memoria: “PID: <PID> - Acción: <LEER / ESCRIBIR> - Dirección Física: <DIRECCION_FISICA> - Valor: <VALOR LEIDO / ESCRITO>”.

Archivo de Configuración

Campo	Tipo	Descripción
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level_from_string()

Ejemplo de Archivo de Configuración

```
LOG_LEVEL=INFO
```

Módulo: Memory Stick

El módulo Memory Stick representará un espacio de direcciones de memoria simulando los diferentes chips de memoria que se tienen en una computadora.

Lineamiento e Implementación

El módulo **Memory Stick** es el encargado de atender las diferentes peticiones de lectura y escritura de las CPUs o del Kernel Memory. Para ello, primero deberá conocer su tamaño, el cual deberá ser ingresado como un parámetro al iniciar el módulo**.**

```
➜ ~ ./bin/memory_stick [Archivo Config] [Tamaño]
```

Una vez leído el tamaño de memoria que representa este Proceso, reservará con `malloc()` dicha cantidad de memoria y deberá conectarse al Kernel Memory. En caso de que la conexión con Kernel Memory sea exitosa, deberá quedarse a la espera de las conexiones de las diferentes CPUs para que estas le hagan pedidos de lectura y escritura sobre la memoria reservada con `malloc()`.

Operaciones

El módulo Memory Stick, solamente podrá responder a 2 operaciones que consisten en la escritura o lectura de los datos. En todos los casos las direcciones físicas arrancarán en 0 para el Memory Stick en cuestión.

Escritura de datos

Para esta operación se recibirá la dirección física a partir de la cual se deberá escribir y el contenido a escribir. Los grupos podrán añadir información extra para facilitar la implementación de esta operación.

Como resultado de esta operación deberán devolver una confirmación al módulo que los haya llamado.

Lectura de datos

El módulo Memory Stick recibirá una dirección física y un tamaño a leer. Como resultado de esta operación deberá devolver los bytes leídos.

Logs mínimos y obligatorios

Conexión a Kernel Memory: "`## Conectado a Kernel Memory`"

Conexión de CPU: "`## CPU <ID CPU> Conectada`"

Escritura: "`## Escritura de <CANTIDAD ESCRITA> bytes`"

Lectura: "`## Lectura de <CANTIDAD ESCRITA> bytes`"

Archivo de Configuración

Campo	Tipo	Descripción
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level_from_string()
MEMORY_DELAY	Número	Tiempo en milisegundos que se debe esperar antes de contestar una lectura o escritura

Ejemplo de Archivo de Configuración

```
LOG_LEVEL=INFO
MEMORY_DELAY=1500
```

Módulo: IO

El Módulo de IO será el encargado de simular las posibles IO que existen en el sistema, siendo estas de 3 tipos: STDIN, STDOUT y SLEEP.

Lineamiento e Implementación

El módulo **IO** es el encargado de simular las operaciones de Entrada/Salida, para ello al iniciar deberá leer como parámetro de entrada el nombre el cual identificará a la interfaz de IO.

```
➜ ~ ./bin/io [Archivo Config] [Tipo]
```

Una vez leído el tipo de IO, deberá conectarse al Kernel Scheduler y quedarse a la espera de las peticiones que este le pueda hacer. Al momento de recibir la petición de IO el comportamiento será el definido a continuación de acuerdo al tipo de IO recibido por parámetro al iniciar.

STDIN

Recibirá una cantidad de caracteres (bytes) que deberá leer por teclado y se quedará esperando el input del usuario, el cual finalizará cuando éste pulse la tecla `Enter`. En caso de que la cadena de caracteres sea mayor al tamaño recibido, se deberá cortar el contenido a la cantidad de bytes solicitada. En caso de que sea menor, se completará con valores `'\0'` hasta alcanzar el límite indicado para devolver dicha información al Kernel Scheduler.

STDOUT

Recibirá una cadena de caracteres (bytes) la cual deberá imprimir por pantalla y en el archivo de Log. Una vez impreso el log deberá informar al Kernel que la IO finalizó correctamente.

SLEEP

Recibirá un tiempo en milisegundos como parámetro adicional para ejecutar un `usleep()`[3] de dicho tiempo, antes de contestarle al Kernel Scheduler que la IO finalizó correctamente.

Logs mínimos y obligatorios

Conexión a Kernel Scheduler: "`## Conectado a Kernel Scheduler`"

Inicio de IO: "`## PID: <PID> - Inicio de IO`"

Fin de IO: "`## PID: <PID> - Fin de IO`"

Solo para STDOUT: "`## PID: <PID> - <CONTENIDO A IMPRIMIR>`"

Solo para STDIN: "`## PID: <PID> - Ingrese <CANTIDAD A LEER> caracteres:`"

---

[3] Tener en cuenta que la función usleep() espera microsegundos.

Solo para SLEEP: “`## PID: <PID> - Haciendo sleep por <TIEMPO> milisegundos.`”

Archivo de Configuración

Campo	Tipo	Descripción
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level_from_string()

Ejemplo de Archivo de Configuración

```
LOG_LEVEL=INFO
```

Módulo: SWAP

El módulo SWAP será el encargado de guardar la información de los Procesos que sean suspendidos por el Kernel Scheduler cuando el módulo Kernel Memory se lo indique.

Lineamiento e Implementación

Para poder almacenar la memoria de los Procesos suspendidos, el módulo SWAP contará con un único archivo cuyo path y tamaño serán definidos por los parámetros SWAP_FILE_PATH y SWAP_FILE_SIZE del archivo de configuración. El archivo que contiene la información de los Procesos se asume que inicia siempre libre, no siendo necesario que su contenido se tenga que limpiar explícitamente.

```
➜ ~ ./bin/swap [Archivo Config]
```

Este archivo se dividirá internamente en bloques de un tamaño definido también por archivo de configuración (BLOCK_SIZE).

Al iniciar, el módulo deberá informar al módulo Kernel Memory el tamaño de bloque y el tamaño total del SWAP.

Dado que la administración del espacio de SWAP es tarea del Kernel Memory, este módulo no deberá contar con ninguna estructura administrativa más allá de lo necesario para leer y/o escribir el archivo utilizado para persistir la información.

Operaciones

El módulo SWAP solamente podrá responder a 2 operaciones que consisten en la escritura o lectura de los datos. En ambos casos las operaciones siempre tendrán el tamaño de 1 bloque.

Escritura de bloque

Para esta operación se recibirá el número de bloque a escribir y su contenido asociado.

Como resultado de esta operación se deberá devolver una confirmación al Kernel Memory de que la escritura fue exitosa.

Lectura de bloque

Para esta operación el módulo SWAP recibirá el número de bloque a leer y como resultado de esta operación deberá devolver los bytes leídos.

Logs mínimos y obligatorios

Conexión a Kernel Memory: “`## Conectado a Kernel Memory`”

Escritura: “`## Escritura del bloque: <NUMERO_BLOQUE>`”

Lectura: "`## Lectura del bloque: <NUMERO_BLOQUE>`"

Archivo de Configuración

Campo	Tipo	Descripción
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level_from_string()
SWAP_FILE_PATH	String	Path al archivo de SWAP
SWAP_FILE_SIZE	Numérico	Tamaño en Bytes del archivo de SWAP
BLOCK_SIZE	Numérico	Tamaño en Bytes de cada bloque del archivo de SWAP

Ejemplo de Archivo de Configuración

```
LOG_LEVEL=INFO
SWAP_FILE_PATH=/tmp/tp_so_swapfile.bin
SWAP_FILE_SIZE=2048
BLOCK_SIZE=64
```

Descripción de las entregas

Debido al orden en que se enseñan los temas de la materia en clase, los checkpoints están diseñados para que se pueda realizar el trabajo práctico de manera iterativa incremental tomando en cuenta los conceptos aprendidos hasta el momento de cada checkpoint.

Check de Control Obligatorio 1: Conexión inicial

Fecha: 18/04/2026

Objetivos:

- Comprender la arquitectura y la comunicación entre los módulos del sistema.
- Todos los módulos están creados y son capaces de establecer conexiones entre sí mediante sockets.

Propósito académico:

- Familiarizarse con el entorno de desarrollo y el repositorio.
- Aprender a utilizar las Commons, principalmente las funciones para listas, archivos de configuración y logs.

Check de Control Obligatorio 2: Planificación

Fecha: 23/05/2026

Objetivos:

- **Módulo Kernel Scheduler:**
  - Planificación de corto y largo plazo con FIFO y RR.
  - Administración del estado BLOCK y las colas de los módulos de IO.
  - Administración de los Mutex y sus colas de Procesos sin herencia de prioridades.
- **Módulo CPU:**
  - Realiza el ciclo de instrucción completo siendo capaz de interpretar y ejecutar instrucciones (Sin memoria).
- **Módulo Kernel Memory:**
  - Devolver lista de instrucciones.
  - Devolver un valor fijo de espacio libre (mock).
  - Contestar OK a lecturas y escrituras de memoria sin implementarlas.
- **Módulo IO: Completo**

Check de Control Obligatorio 3: CPU Completa y Memoria

Fecha: 20/06/2026

Objetivos:

- **Módulo Kernel Scheduler:**
  - Planificación de corto y largo plazo completa.
  - Implementación de Herencia de Prioridades.

- **Módulo CPU: Completo**

- **Módulo Kernel Memory:**
  - Esquema de Memoria principal completo.
  - Administración de espacio libre en memoria principal.
  - Lectura y escritura de Memory Sticks.

- **Memory Stick: Completo**

Entregas Finales

Fechas: 11/07/2026, 18/07/2026, 01/08/2026

Objetivos:

- Finalizar el desarrollo de todos los módulos.
- Probar de manera intensiva el TP en un entorno distribuido.
- Todos los componentes del TP ejecutan los requerimientos de forma integral.